



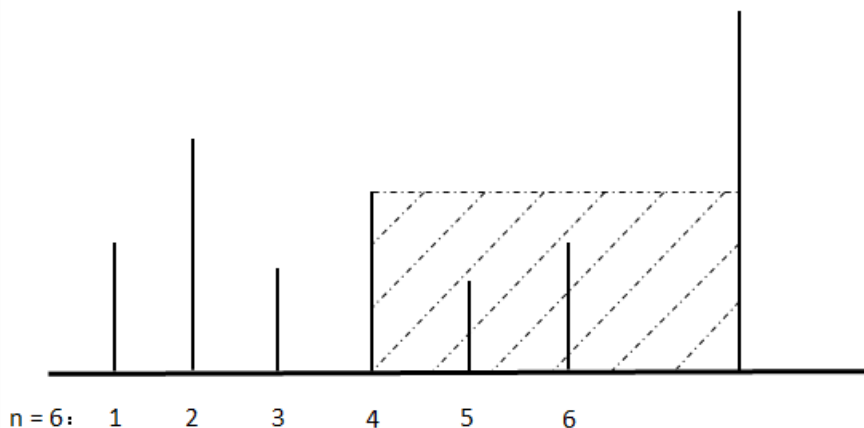
# 单调栈

栈（stack）是指一种先入后出（FILO）的结构，有一种特殊的栈——单调栈。顾名思义，单调栈就是指栈内的元素单调递增或者单调递减的一种结构。他常用来优化某些问题的时间复杂度，达到优化效率的效果。



# 木板倒水

地上从左到右竖立着  $n$  块木板，从 1 到  $n$  依次编号，如下图所示。我们知道每块木板的高度，在第  $n$  块木板右侧竖立着一块高度无限大的木板，现对每块木板依次做如下的操作：对于第  $i$  块木板，我们从其右侧开始倒水，直到水的高度等于第  $i$  块木板的高度，倒入的水会淹没  $a_i$  块木板（如果木板左右两侧水的高度大于等于木板高度即视为木板被淹没），求  $n$  次操作后，所有  $a_i$  的和是多少。如图上所示，在第 4 块木板右侧倒水，可以淹没第 5 块和第 6 块一共 2 块木板， $a_4 = 2$ 。（注意要，该块木板的左右两边都被水淹没才算是木板被淹没）



输入：

5

10 5 8 12 6

$n \leq 1000000$

输出：

3

# 木板倒水

这道题直接做很简单，遍历 $n$ 根柱子，找到每根柱子右边第一个高度不小于他的柱子就表示寻找结束，中间的就是可以淹没的。尝试 每根柱子的时间复杂度是 $O(n)$ ，对于每一根柱子往右边依次寻找时间复杂度也是 $O(n)$ ，所以总的时间复杂度是 $O(n^2)$ ，然后数据范围是 $100w$ ，很明显，过不了。

我们再仔细来分析一下这道题。对于第 $i$ 根柱子。

(1) 如果下一根柱子高度 $<h[i]$ ，说明可以淹没，然后继续往后面找，直到找到第 $k$ 根柱子高度 $\geq h[i]$ ，我们再仔细想一下，第 $k$ 根柱子是不是也是 $[i, k)$ 之间的柱子的终止柱子。所以完全不需要再去计算一遍。

# 木板倒水

假设 $n=5$ : 10 5 8 12 6

(1) 对于第一根柱子 (10), 由于此时我们不知道后面柱子高度, 所以无法判断他的终止柱子 (第一个不可以淹没的柱子)

(2) 对于第二根柱子 (5),  $10 < 5$ , 说明往第一根柱子倒水肯定可以淹没第二根柱子, 但是前两根柱子的终止柱还是不知道。

(3) 对于第三根柱子 (8),  $5 < 8$ , 说明第二根柱子的终止柱出现了, 淹没柱子为  $3 - 2 - 1 = 0$ , 此时1, 3根柱子的终止柱未知。

(4) 对于第四根柱子 (12),  $8 < 12$ ,  $10 < 12$ , 说明第1, 3根柱子的终止柱出现了, 虽然  $5 < 12$ , 但是在前面5的终止柱已经找到了, 就不再考虑了, 淹没柱子为  $(4 - 3 - 1) + (4 - 1 - 1) = 2$ .

# 木板倒水

(5) 此时未找到终止柱的只有4号柱(12)，然后对于第五根柱子(6)，此时4,5号柱子的终止柱都没有出现，继续往后面遍历，题目中告诉了，最右边有一根无穷大的柱子，即第6根柱子为正无穷，所以此时第4,5根柱子的终止柱出现了，所以淹没柱子为， $(6-5-1)+(6-4-1)=1$ ；

(6) 把前面所有淹没柱子数量加起来 $0+2+1=3$ . 即为最终结果。

对于前面的分析，我们发现，其实就是找当前未找到终止柱的柱子的终止柱，如果找到了，两者位置相减就是淹没的柱子，同时，对于已经找到终止柱的柱子就不需要再考虑了。如果下一根柱子高度 $<$ 当前柱子高度，说明需要继续往下面找，否则就表示找到终止柱了，同时，我们比较的时候肯定是先比较最后一根柱子和当前柱子的高度。来判断是否该继续寻找还是已经找到。

# 木板倒水

假设 $n=6$ : 6 8 4 7 3 7

(1)  $h[1]=6$ , 栈为空, 入栈。编号(num)和高度(h)

|    |   |  |  |  |  |  |  |
|----|---|--|--|--|--|--|--|
| 编号 | 1 |  |  |  |  |  |  |
| 高度 | 6 |  |  |  |  |  |  |

# 木板倒水

(2)  $h[2]=8$ , 比较和栈顶元素的大小,  $s.top < h[2]$ , 说明栈顶柱的终止柱就找到了, 算出淹没柱子的根数  $2-1-1=0$ . 找到终止柱之后就不需要再找了, 出栈。继续判断栈顶高度和当前高度的大小。栈为空, 入栈。如果此时栈顶元素高度仍小于当前高度, 继续出栈。

编号

2

高度

8

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |

# 木板倒水

(3)  $h[3]=4$ ,  $s[\text{top}]>h[3]$ , 虽然表示往上一根柱子倒水可以淹没该柱子, 但是我们不能确定还能不能继续淹没后面的柱子, 所以, 都保留, 入栈。

栈内保留的都是未找到终止柱的柱子。

|    |   |   |  |  |  |  |  |  |
|----|---|---|--|--|--|--|--|--|
| 编号 | 2 | 3 |  |  |  |  |  |  |
| 高度 | 8 | 4 |  |  |  |  |  |  |



# 木板倒水

(4)  $h[4]=7$ ,  $s[\text{top}]<h[3]$ , 表示当前栈顶柱子找到了终止柱, 出栈, 计算淹没的柱子根数  $4-3-1=0$ . 继续判断栈顶柱子和当前柱子高度,  $s[\text{top}]>h[4]$ , 入栈。寻找终止柱

编号

高度

|   |   |  |  |  |  |  |  |
|---|---|--|--|--|--|--|--|
| 2 | 4 |  |  |  |  |  |  |
| 8 | 7 |  |  |  |  |  |  |

# 木板倒水

(5)  $h[5]=3$ ,  $s[\text{top}]>h[5]$ , 可以淹没, 终止柱未知, 入栈。

|    |   |   |   |  |  |  |  |  |
|----|---|---|---|--|--|--|--|--|
| 编号 | 2 | 4 | 5 |  |  |  |  |  |
| 高度 | 8 | 7 | 3 |  |  |  |  |  |

# 木板倒水

(6)  $h[6]=7$ ,  $s[top]<h[6]$ , 栈顶柱子的终止柱找到, 出栈并计算淹没的柱子,  $6-5-1=0$ , 继续判断  $s[top]$  和  $h[6]$  的大小,  $s[top]==h[6]$ , 也表示终止柱找到了, 出栈,  $6-4-1=1$ , 表示淹没了一根柱子, 继续判断  $s[top]>h[6]$ , 入栈。

编号

高度

|   |   |  |  |  |  |  |  |
|---|---|--|--|--|--|--|--|
| 2 | 6 |  |  |  |  |  |  |
| 8 | 7 |  |  |  |  |  |  |

# 木板倒水

(7) 因为最右边有一根无穷高的柱子，所以我们可以让  $h[6+1]=\max.$ ， $s[\text{top}]<h[6+1]$ ，出栈并计算  $7-6-1=0$ 。  $s[\text{top}]<h[6+1]$ ，出栈并计算  $7-2-1=4$ 。此时栈为空， $h[7]$  入栈，程序结束。前面计算的时候没出栈一次，计数器累加一次。最终得到淹没柱子的总数。

|    |     |  |  |  |  |  |  |
|----|-----|--|--|--|--|--|--|
| 编号 | 7   |  |  |  |  |  |  |
| 高度 | max |  |  |  |  |  |  |

# 木板倒水

我们来大概算一下时间复杂度，所有柱子均会入栈一次（入栈等待终止柱的到来），当遇到终止柱的时候出栈一次，最后栈为空。所以时间复杂度就是 $O(n)$ 。

总的来说，如果不确定当前柱子的终止柱就入栈，找到当前栈顶柱的终止柱就出栈并且计算淹没的柱子，这样做就不会有重复计算的部分。所以效率会非常高。这道题使用的栈内元素都是单调递减的，所以叫单调递增栈（从栈顶到栈顶）。

# 木板倒水

if(栈为空) 当前柱子入栈;

else

{

if(高度<栈顶高度)

当前柱子入栈;

else

{

出栈, 直到高度 $\geq$ 栈顶高度  
或者栈为空;

记录淹没柱子数并且累加;

入栈;

}

当栈为空的时候,  
无法取栈顶元素,  
该操作是错误的

当高度相等时, 也表示终止了, 因为水灌到柱子高度就停止

}

# 最长不下降子序列

已知一个长度为 $n$ 的数列，求数列中的最长不下降子序列的长度。在序列中，对于所有的 $i < j$ ,  $h[i] \leq h[j]$ 。

输入：

7

1 3 6 8 4 5 7

输出：

5

数据范围：  $n \leq 100000$

最长不下降子序列是1 3 4 5 7，长度为5

# 最长不下降子序列

对于 $n=7$  ; 1 3 6 8 4 5 7

如果先不考虑数据范围，用常规的办法求最长不下降子序列。

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| 高度   | 1 | 3 | 6 | 8 | 4 | 5 | 7 |
| 序列长度 | 1 | 2 | 3 | 4 | 3 | 4 | 5 |

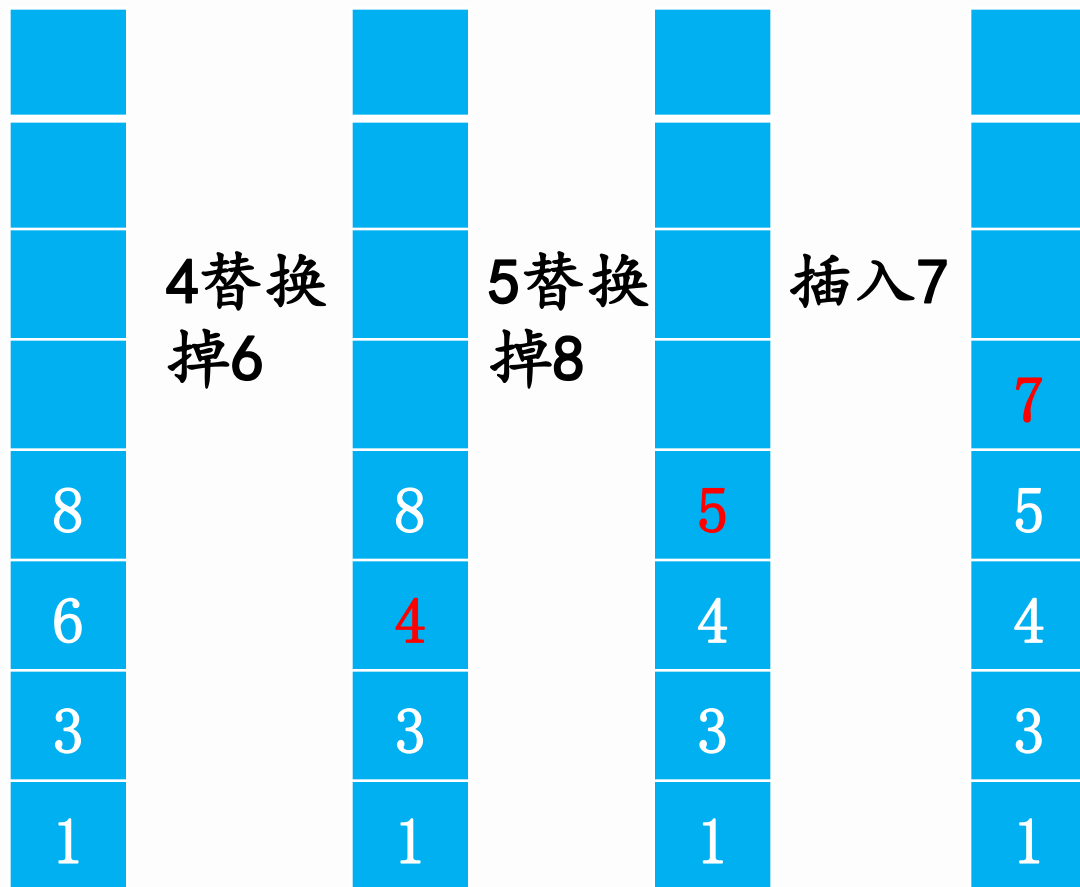
对于每一个高度，寻找他前面高度不高于他的位置，找出其中序列长度最大的，然后+1就是当前最长不下降子序列的长度。对于每一个位置，需要把前面遍历一遍，时间复杂度为 $O(n^2)$ 。



# 最长不下降子序列

这道题我们可以先用单调栈来做。

对于  $n=7$  ; 1 3 6 8 4 5 7



最大长度就是5

# 最长不下降子序列

这样看上去，虽然效率上有一点优化，但是并没有太大的提升，时间复杂度仍然还是 $O(n^2)$ ；但是既然这是个单调栈，那就说明这个栈具有单调性，而我们要在具有单调性的序列中去搜索一个数，很明显可以用二分查找来优化效率，所以总的时间复杂度可以变为 $O(n \log n)$ 。

```
if(h[i]>=s[top])
```

```
{
```

入栈；

```
}
```

```
else
```

```
{
```

二分查找更新的位置

//最后一个比他小的数后面

```
}
```

因为是不下降子序列，所以栈内可能有重复元素，所以二分查找的时候要注意。

更新的意思是，相同长度的序列，让该位置的值最小，这样便于后面的插入。



# 木板倒水

